Supplement Recommendation Engine Technical Handover Document

Architecture, functions, persistence, logging, and maintenance guide for engineers taking over the Docker pilot build.

Status: Phase 1 → 2c-M2 complete (pilot-ready) | **Repo root:** supplement_engine/ | **Orchestrator:** python scripts/run_app.py

1. Big picture overview

The engine maps patient state + clinical evidence into ranked nutraceutical recommendations behind a deterministic safety gate. At score time it **only** reads **PostgreSQL** (patient realm) and **Neo4j** (knowledge graph). It never queries an upstream warehouse. Bulk load / real-time LIS ingest are delegated to an external feeder project that writes Postgres tables.

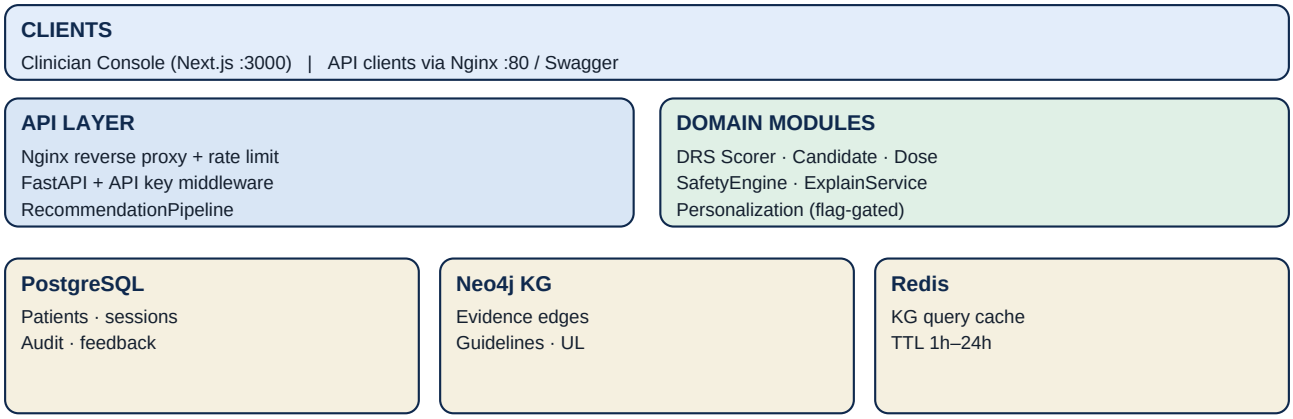

Figure 1 — Logical architecture layers

1.1 Request path (happy path)

Client → Nginx (:80) → FastAPI middleware (request ID, optional API key, JSON logs) → **POST /v1/recommendations** → PatientRepository → ProfileValidator → RecommendationPipeline → persist session / audit / evidence snapshot → response JSON. The clinician console never calls the engine from the browser with the raw key; Next.js route handlers proxy with **ENGINE_API_KEY**.

1.2 Docker services (default pilot)

Service	Port	Purpose
api	8000	FastAPI engine + Alembic migrate on boot
frontend	3000	Next.js clinician console (standalone)
nginx	80/443	Reverse proxy, rate limit /v1/*
postgres	5432	Patients, sessions, audit, feedback
neo4j	7474/7687	Knowledge graph (no PHI)
redis	6379	KG read cache
kafka + zk	9092	Optional; compose profile **ingestion**

Kafka is off by default (**KAFKA_ENABLED=0**). Enable later with **docker compose --profile ingestion up -d** when event streaming is needed.

2. Scoring pipeline — functions in order

RecommendationPipeline.evaluate() — left to right

Figure 2 — RecommendationPipeline stage order

Stage	Module / class	Purpose
Load	PatientRepository ProfileValidator	Load patient by UUID (or inline in dev). Drop invalid ICD-10 / RxNorm / LOINC codes.
1 DRS	DeficiencyRiskScorer	Bayesian log-odds risk per nutrient: baseline, geo, BMI, conditions, meds, labs.
1b	PersonalizationEngine	Optional blend with prior drs_snapshot (PERSONALIZATION_ENABLED=1).
2	CandidateGenerator	Keep nutrients above DRS threshold or guideline triggers; collapse B-complex.
3	DoseOptimizer	Pregnancy/guideline/RDA/bioavailability/BMI; soft UL cap ~70%.
4	SafetyEngine	Deterministic: drug–nutrient, disease CI, nutrient–nutrient, UL hard block, escalation.
5	ConfidenceCompositor	Composite confidence; rank = P_deficient × confidence × I_safety.
6	ExplainService	Template fill → rationale.why / .evidence / .safety.
7	EvidenceSnapshot	Capture kg version + content_hash for reproducibility.
Persist	Repositories + Kafka*	Write session, recommendations, audit; optional recommendation.served event.

2.1 Key source files

Concern	Path
HTTP API / schemas	src/api/app.py
Pipeline orchestration	src/pipelines/recommendation_pipeline.py
DRS	src/core/deficiency_risk_scorer.py
Candidates + dose + confidence	src/core/candidate_generator.py
Safety gate	src/safety/safety_engine.py
Explain	src/explain/explain_service.py
Neo4j access + Redis	src/knowledge/graph_client.py
ORM + repositories	src/db/orm_models.py, src/db/repositories.py
Domain models	src/shared/models.py
Personalization	src/personalization/engine.py
Kafka producer	src/pipelines/kafka_producer.py
Console UI	frontend/app/page.tsx + components/*
Console API proxy	frontend/lib/engine-proxy.ts, frontend/app/api/*/route.ts
Stack orchestrator	scripts/run_app.py
Patient DB contract	etl/PATIENT_REALM_CONTRACT.md

3. API surface

3.1 Ops

Method	Path	Notes
GET	/health	Neo4j + Postgres aggregate
GET	/health/live	Liveness
GET	/health/ready	Readiness (503 if deps down)

3.2 Recommendations & patients

Method	Path	Notes
POST	/v1/recommendations	Score; prod uses {patient_id} only
GET	/v1/sessions/{id}	Stored session
GET	/v1/patients/{id}/history	Recent sessions
POST	/v1/patients/{id}/labs	Append lab (delta)
POST	/v1/patients/{id}/medications/sync	Replace active meds
POST	/v1/patients/{id}/conditions/sync	Upsert conditions
POST	/v1/feedback	Clinician override
GET	/v1/audit/{session_id}	Audit + input hash
GET	/v1/evidence/{snapshot_id}	KG snapshot at serve
GET	/v1/safety/check	Standalone interaction check
GET	/v1/nutrients/{id}	Nutrient metadata

Interactive docs: **http://localhost/docs** (via Nginx) or :8000/docs.

3.3 Frontend proxy contract

Browser → **/api/recommendations**, **/api/sessions/...**, **/api/patients/.../history**, **/api/feedback** → Next route handler → **ENGINE_API_URL** + header **X-API-Key: ENGINE_API_KEY**. Types in **frontend/lib/types.ts** must stay aligned with FastAPI Pydantic models.

4. Data stores & contractual boundaries

4.1 PostgreSQL (PHI + engine output)

- **Patient realm:** demographics, conditions, medications, labs (source of scoring truth).
- **Engine output:** recommendation sessions, per-nutrient rows, suppressed reasons.
- **Audit:** append-only audit_log with input hash; evidence_snapshots with content_hash.
- **Feedback:** clinician approve / adjust / reject events.
- **Migrations:** Alembic runs on API container start (docker-entrypoint.sh).

External feeder must write the same tables; never ask the engine to query a warehouse. Contract details: **etl/PATIENT_REALM_CONTRACT.md**.

4.2 Neo4j (no PHI)

Nutrients, ICD-10, RxNorm, LR edges, guidelines, baselines, interactions. Accessed only via GraphClient. Seed file: **scripts/neo4j_seed.cypher**. Cacheable reads go through Redis; interaction/safety edges are **not** cached.

4.3 Logging & tracking (for handover / ops)

Signal	Where / what
Request ID	Middleware stamps every HTTP request; correlate API logs.
JSON access logs	No raw PHI; patient_id hashed when logged (Phase 2b M4).
session_id	Returned on every score; key for audit + history.
evidence_snapshot_id	Points to KG content hash at serve time (reproducibility).
model_version	Loaded at API startup; stored on session.
feedback_id	Returned by POST /v1/feedback.
Docker logs	docker compose logs api --tail 100 (also frontend, nginx)
Health	GET /health and /health/ready for probes.
Validation gates	scripts/validate_phase*.ps1 for regression proof

5. Clinician console (frontend)

Next.js 14 App Router, TypeScript, Tailwind tokens in **globals.css**. Docker image uses **output: standalone**.

Component	Responsibility
page.tsx	Orchestrates intake → pipeline animation → results / history
IntakeForm	Stored UUID cohort + inline demography/conditions/meds/labs
SafetyPipeline	Five-gate visual during scoring
ResultsPanel / SessionLedger	Session metadata, banners, suppressed, cards
RecommendationCard	Dose, meters, gates, rationale tabs, feedback buttons
SessionHistoryPanel	Prior sessions for a patient
ThemeToggle	Light / dark persisted theme
Toast	Transient feedback for copy/save actions

6. Operate & maintain

6.1 Day-1 commands

```
python scripts/run_app.py up --open
python scripts/run_app.py status
python scripts/run_app.py smoke
python scripts/run_app.py seed --all --force
python scripts/run_app.py down
```

6.2 Environment flags that matter

Variable	Effect
ALLOW_INLINE_PATIENT	1=dev inline body OK; 0=prod patient_id only
REQUIRE_API_KEY / API_KEYS	Prod overlay enforces X-API-Key on /v1/*
KAFKA_ENABLED	0 by default; producers no-op when off
PERSONALIZATION_ENABLED	Stage 1b blend of prior DRS snapshots
ENGINE_API_URL / ENGINE_API_KEY	Frontend container → api
DRS_THRESHOLD / MIN_CONFIDENCE	Tuning knobs for candidate / confidence floors

6.3 Maintenance playbook

- **Schema change:** add Alembic revision under alembic/versions/; restart api (auto-migrate).
- **KG evidence change:** update scripts/neo4j_seed.cypher or registry YAMLs + compile; re-seed with --force; confirm evidence content_hash changes on next score.
- **New safety rule:** edit SafetyEngine (+ unit test); never soft-cache interaction edges.
- **API contract change:** update Pydantic in app.py and frontend/lib/types.ts together; rebuild frontend image.
- **Stuck containers:** docker compose down --remove-orphans then python scripts/run_app.py up --no-build.
- **Regression:** run validate_phase2b_prod_gate.ps1, pilot, phase2c gates before a demo.
- **Secrets:** rotate API_KEYS / CONSOLE_AUTH_* for any environment with real PHI.

7. Tests & quality gates

Layer	How
Unit (no Docker)	pytest tests/unit/ -v
Integration	Stack up + pytest tests/integration/ -m integration
Phase gates	scripts/validate_phase1_gate.ps1 … validate_phase2c_m2_gate.ps1
Frontend	cd frontend && npm run typecheck && npm run build
Smoke	python scripts/run_app.py smoke

8. Known gaps / deferred work

- Bulk warehouse feeder (external repo) + joint integration smoke.
- Kafka consumers / analytics mart / FHIR lab parser.
- Console UI for audit/evidence and lab-delta-then-rescore (APIs exist).
- Feedback → nightly retrain DAG; model A/B benchmark CLI.
- Phase 3: SMART-on-FHIR, LightGBM dose-response (±30% rules bound).
- Phase 4: SFDA SaMD package, PGx, payer HEOR.

9. Handover checklist

- Can start full stack with run_app.py and open console + Swagger.
- Can score pilot UUID and recover session via GET /v1/sessions/{id}.
- Can fetch audit + evidence for a session and verify content_hash is stable for same KG.
- Knows ALLOW_INLINE_PATIENT must be 0 in production profiles.
- Knows Kafka is optional (compose profile ingestion).
- Knows patient bulk load is out-of-repo; uses PATIENT_REALM_CONTRACT.md.
- Has run at least phase2b prod + pilot validation gates once.
- Has access to ENGINE_MASTER_REFERENCE.md and ENGINE_DIAGRAMS.html for deeper diagrams.

Primary references in-repo: ENGINE_MASTER_REFERENCE.md · ENGINE_DIAGRAMS.html · etl/PATIENT_REALM_CONTRACT.md · README.md · frontend/README.md